Project Terminus — Complete Knowledge Guide

Table of Contents

What is Project Terminus?

Project Terminus is a **benchmark dataset for AI coding agents** created by Snorkel AI. Think of it as a "test suite" to measure how well AI models (like GPT-4, Claude, etc.) can solve real-world programming tasks.

Simple Analogy

Imagine you're creating a puzzle box:

- **You** (the task creator) build the puzzle with hidden challenges
- **An AI agent** (like GPT-5 or Claude) tries to solve it
- **The test suite** checks if the AI solved it correctly
- **Snorkel AI** uses the results to understand which AI models are best at coding

---

Purpose and Goal

The Big Picture

Snorkel AI needs hundreds of diverse coding tasks to test AI agents. Your job is to create **one task** that:

1. Tests a specific coding skill (debugging, data processing, etc.)
2. Is challenging enough that AI agents don't solve it 100% of the time
3. Has a clear way to verify if the agent succeeded

Your Specific Task

You created: **"csv-region-aggregator"** — a debugging task where an AI agent must:

- Find and fix multiple bugs in a broken Python script
- Make it aggregate sales data from a CSV file
- Output correct JSON with regional totals

What Success Looks Like

- ✅ Oracle solution (your perfect fix) passes all tests
- ✅ AI agents (GPT, Claude) solve it ~30-70% of the time (not too easy, not impossible)
- ✅ All quality checks (CI/LLMaJ) pass
- ✅ Peer reviewers approve your submission
- 💰 You get **$250** per accepted task

Requirements

Technical Requirements

Category	Requirement
Docker	All code runs inside a Docker container (isolated environment)
Python version	Python 3.11.4 (pinned with SHA256 digest)
Dependencies	All packages must have exact version numbers (pandas==2.1.0)
Base image	Must use official `python:3.11.4-slim` (Debian-based)
Test framework	pytest 7.4.3 for automated testing
File size	All files must be under 1 MB

Quality Requirements

1. **Task must be solvable** — Your "oracle solution" (solve.sh) must pass all tests every time

2. **Not too easy** — GPT-5.2 and Claude Opus should NOT both solve it 100% of the time
3. **Not too hard** — At least one of them should succeed sometimes
4. **Anti-cheat** — Tests must verify actual computation, not hardcoded answers
5. **Well-documented** — Clear instructions, docstrings, comments

Submission Requirements

- Pass all **CI checks** (Dockerfile format, file structure, paths)
- Pass all **LLMaJ checks** (instruction quality, test coverage, no cheating)
- Pass **peer review** (human expert reviews your task)
- Get accepted **difficulty rating** (not "Trivial")

Project Structure

```
PROJECT TERMINUS/
├── environment/              ← The "puzzle box" the AI agent sees
│   ├── Dockerfile            ← Builds the Docker container
│   ├── aggregator.py         ← The BROKEN Python script (has 4 bugs)
│   └── .dockerignore         ← Files Docker should ignore
│
├── solution/                 ← Your perfect solution (hidden from AI)
│   └── solve.sh              ← Shell script that fixes all bugs
│
├── tests/                    ← Automated verification (hidden from AI)
│   ├── test.sh               ← Test runner script
│   └── test_outputs.py       ← 10 pytest tests to verify correctness
│
├── instruction.md            ← What the AI agent reads (the task
description)
├── task.toml                 ← Metadata: task name, tags, runtime limits
├── .gitignore                ← Git ignore rules
└── PROJECT_KNOWLEDGE.md      ← This file (documentation)
```

File-by-File Explanation

1. `environment/Dockerfile`

Purpose: Builds the Docker container that the AI agent runs inside.

Line-by-line breakdown:

```
# Line 1: Base image — Python 3.11.4 on Debian "slim" variant
# The @sha256:... ensures we always get the EXACT same image
FROM python:3.11.4-slim@sha256:17d62d681d9ecef20aae6c6605e9cf83b0ba3dc247

# Line 3: Prevent interactive prompts during apt-get install
ENV DEBIAN_FRONTEND=noninteractive

# Lines 6-8: Install system package "curl" with exact version
# --no-install-recommends = don't install suggested packages (keeps image
# rm -rf /var/lib/apt/lists/* = delete package cache to save space
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl=7.88.1-10+deb12u14 \
 && rm -rf /var/lib/apt/lists/*

# Lines 11-14: Install Python packages with EXACT versions
# pandas = data analysis library (used to read CSV files)
# numpy = numerical computing library (pandas depends on it)
# pytest = testing framework (used in tests/test.sh)
RUN pip install --no-cache-dir \
    pandas==2.1.0 \
    numpy==1.26.0 \
    pytest==7.4.3

# Line 16: Set working directory inside container to /app
WORKDIR /app

# Line 19: Create folders for data and output
RUN mkdir -p /app/data /app/output

# Lines 22-46: Generate the sales.csv file with 12 rows of fake sales dat
# Uses Python's csv module to write:
#   id,region,product,amount
#   1,north,widget,120.50
#   ... (10 more rows)
RUN python3 -c "\
import csv; \
rows = [ \
    ['id', 'region', 'product', 'amount'], \
    ['1',  'north',  'widget',  '120.50'], \
    # ... (data rows)
]; \
f = open('/app/data/sales.csv', 'w', newline=''); \
```

```
w = csv.writer(f); \
[w.writerow(r) for r in rows]; \
f.close(); \
print('sales.csv created') \
"

# Lines 49-50: Copy the BROKEN aggregator.py script into the container
COPY environment/aggregator.py /app/aggregator.py

# Lines 53-59: Verify both files exist (will fail build if missing)
RUN python3 -c "\
import os; \
assert os.path.exists('/app/aggregator.py'), 'aggregator.py missing'; \
assert os.path.exists('/app/data/sales.csv'), 'sales.csv missing'; \
print('Environment verified') \
"
```

What it does: Creates a container with Python, pandas, a CSV file, and a broken Python script. This is the "starting state" the AI agent sees.

---

2. environment/aggregator.py

Purpose: The BROKEN Python script the AI agent must fix.

Line-by-line breakdown:

```
# Lines 1-3: Import required libraries
import pandas as pd  # For reading CSV and data manipulation
import json          # For writing JSON output
import os          # For file path operations

# Lines 5-6: Define file paths as constants
DATA_FILE   = '/app/data/sales.csv'      # Input CSV
OUTPUT_FILE = '/app/output/summary.json' # Output JSON

# Lines 9-11: Helper function to parse amount values
def _parse_amount(value):
    """Parse a sales amount value, stripping any whitespace."""
    return float(str(value).strip())
    # Converts to string, removes spaces, converts to float

# Lines 14-16: Helper function to normalize region names
```

```
def _normalise_region(name):
    """Return a normalised region name for grouping."""
    return str(name).lstrip()
    # ⚠️ BUG 2: Uses lstrip() instead of strip()
    # lstrip() only removes LEFT whitespace, not right
    # Should be: return str(name).strip()

# Lines 19-42: Main aggregation function (contains multiple bugs)
def aggregate():
    # ⚠️ BUG 3: Forces 'amount' column to be read as strings
    # This causes issues with numeric operations later
    df = pd.read_csv(DATA_FILE, dtype={'amount': str})

    # Apply the helper to convert amounts to float
    df['amount'] = df['amount'].apply(_parse_amount)

    # Apply region normalisation
    df['region_key'] = df['region'].apply(_normalise_region)

    # ⚠️ BUG 1: Uses 'Region' (capital R) but CSV has 'region' (lowercase
    # This will crash with: KeyError: 'Region'
    # ⚠️ BUG 4: Uses .mean() instead of .sum()
    # mean() gives average per region, not total
    summary = (
        df.groupby('Region')['amount']
        .mean()  # ⚠️ Should be .sum()
        .to_dict()
    )

    # Create output directory if it doesn't exist
    os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)

    # Write JSON output with 2-space indentation
    with open(OUTPUT_FILE, 'w') as f:
        json.dump(summary, f, indent=2)

    print('Done.')

# Lines 45-46: Run aggregate() when script is executed directly
if __name__ == '__main__':
    aggregate()
```

What it does: Tries to read CSV, group by region, and write JSON — but crashes due to bugs. The AI must fix all 4 bugs.

3. solution/solve.sh

Purpose: Your "oracle solution" — the perfect fix that passes all tests.

Key sections:

```
#!/bin/bash
# Line 1: This is a bash script

set -e          # Exit immediately if any command fails
set -u          # Treat unset variables as errors
set -o pipefail # Catch errors in pipes

# STEP 1: Verify files exist
[ -f "/app/aggregator.py" ] || { echo "ERROR: missing"; exit 1; }

# STEP 2: Diagnose (show CSV structure)
python3 -c "
import pandas as pd
df = pd.read_csv('/app/data/sales.csv')
print('CSV columns:', list(df.columns))
"

# STEP 3: Apply all 4 fixes programmatically
python3 << 'PYEOF'
with open('/app/aggregator.py', 'r') as f:
    src = f.read()

# FIX 1: Change groupby('Region') to groupby('region')
src = src.replace("groupby('Region')", "groupby('region')")

# FIX 2: Change lstrip() to strip()
src = src.replace('return str(name).lstrip()', 'return str(name).strip()'

# FIX 3: Remove dtype={'amount': str}
src = src.replace("pd.read_csv(DATA_FILE, dtype={'amount': str})",
                  "pd.read_csv(DATA_FILE)")

# FIX 4: Change .mean() to .sum()
src = src.replace('.mean()', '.sum()')

# Write fixed version
```

```
with open('/app/aggregator.py', 'w') as f:
    f.write(src)
PYEOF

# STEP 4: Run fixed script
python3 /app/aggregator.py

# STEP 5: Verify output is correct
python3 -c "
import json, math
with open('/app/output/summary.json') as f:
    data = json.load(f)

expected = {'east': 604.0, 'north': 500.5, 'south': 436.25, 'west': 635.7
for r, v in expected.items():
    assert math.isclose(data[r], v, rel_tol=1e-6)
print('✓ All totals correct')
"
```

What it does: Shows HOW to fix each bug step-by-step, runs the fixed script, verifies output matches expected values.

---

4. tests/test.sh

Purpose: Runner script that executes pytest and writes the reward file.

```
#!/bin/bash
set -uo pipefail

# Create logs directory
mkdir -p /logs/verifier

# Run pytest with -rA flag (show All test results)
python -m pytest /tests/test_outputs.py -rA
rc=$?  # Save exit code (0=pass, non-zero=fail)

# Write reward based on result
if [ "$rc" -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt  # Success
else
    echo 0 > /logs/verifier/reward.txt  # Failure
fi
```

```
exit "$rc"  # Exit with pytest's exit code
```

What it does: Runs all 10 tests, writes `1` to reward.txt if all pass, `0` if any fail.

---

5. tests/test_outputs.py

Purpose: 10 automated tests that verify the AI agent's fix is correct.

Test breakdown:

Test #	Function	What it checks
1	`test_output_file_exists`	`/app/output/summary.json` exists
2	`test_output_file_not_empty`	File is not 0 bytes
3	`test_output_is_valid_json`	File contains valid JSON
4	`test_output_is_dict`	JSON root is `{}` not `[]`
5	`test_all_regions_present`	Has north, south, east, west keys
6	`test_region_keys_are_lowercase_strings`	Keys are `"north"` not `"North"`
7	`test_all_values_are_numeric`	Values are floats not strings
8	`test_all_totals_are_positive`	All values > 0
9	`test_totals_match_csv_data`	Values match CSV sum (anti-cheat)
10	`test_aggregator_script_is_fixed`	Script no longer has `groupby('Region')`

Key anti-cheat mechanism:

```
def _compute_expected_totals():
    """Read the CSV and compute expected region totals independently."""
    totals = {}
    with open(CSV_FILE) as f:
        reader = csv.DictReader(f)
        for row in reader:
            region = row["region"].lower()
            amount = float(row["amount"])
            totals[region] = totals.get(region, 0.0) + amount
    return totals
```

This computes expected totals dynamically from the CSV, so agents can't just read hardcoded expected values from the test file.

---

6. instruction.md

Purpose: The task description the AI agent reads — no hidden solutions!

```
# Fix the broken sales aggregator

There is a Python script at `/app/aggregator.py` that reads sales records
from `/app/data/sales.csv`, groups them by region, and is supposed to wri
a summary to `/app/output/summary.json`.

The script is currently broken in multiple ways. When you run it, it cras
immediately with a `KeyError`. Even after fixing the crash, the output it
produces is incorrect — the numbers do not match the actual sales data ir
the CSV. You will need to read the script carefully and fix every bug you

Fix `/app/aggregator.py` so it runs without errors and produces a correct
output file at `/app/output/summary.json`.

The output must be a JSON **object** (not a list or array) where each key
lowercase region name string and each value is a positive **float** repre
the **sum** of the `amount` column for that region. The exact structure i

```json
{
  "east":  604.0,
  "north": 500.5,
  "south": 436.25,
  "west":  635.75}
```
```

You can verify your fix by running:

```
python3 /app/aggregator.py
```

If successful, the script will print `Done.` and exit with code 0.

```
**What it does:** Tells the AI WHAT to fix (not HOW). Describes the
problem, expected output, and verification method.
```

````
---

### 7. `task.toml`
**Purpose:** Metadata about your task for the Terminus platform.

```toml
[task]
name = "mahnoor/csv-region-aggregator"  # Your username/task-name
task_type = "debugging"                  # Category
codebase_size = "minimal"                # 0-20 files
languages = ["python"]                   # Programming languages used
tags = ["csv", "json", "pandas", "multi-bug", "aggregation", "debugging"]
number_of_milestones = 0                 # No multi-stage tasks

[task.runtime_limits]
agent    = 600    # AI has 10 minutes to solve
verifier = 120    # Tests must complete in 2 minutes
build    = 300    # Docker build must finish in 5 minutes

[task.behaviors]
behaviors = [
  "Script at /app/aggregator.py is fixed to handle the lowercase 'region'
column name",
  "Fixed script reads sales data from /app/data/sales.csv without error",
  # ... (12 total behaviors the AI must achieve)
]

[task.output_files]
files = ["/app/output/summary.json"]  # Files tests check for
```
````

What it does: Tells the platform what your task is, how long it should take, and what behaviors to verify.

How It Works

The Full Workflow

```
┌─────────────────────────────────────────────────────────┐
| 1. YOU create the task                                  |
|    - Write broken code (aggregator.py with 4 bugs)      |
|    - Write perfect solution (solve.sh fixes all bugs)   |
```

```
|    - Write tests (test_outputs.py checks correctness)   |
|    - Write instructions (instruction.md describes task)|
└─────────────────────────────────────────────────────────┘

                          ↓

┌─────────────────────────────────────────────────────────┐
| 2. DOCKER builds the environment                        |
|    - Creates container with Python + pandas             |
|    - Installs broken script at /app/aggregator.py       |
|    - Creates CSV data at /app/data/sales.csv            |
└─────────────────────────────────────────────────────────┘

                          ↓

┌─────────────────────────────────────────────────────────┐
| 3. AI AGENT runs inside container                       |
|    - Reads instruction.md                               |
|    - Explores /app/aggregator.py (sees bugs)            |
|    - Attempts to fix the code                           |
|    - Runs: python3 /app/aggregator.py                   |
└─────────────────────────────────────────────────────────┘

                          ↓

┌─────────────────────────────────────────────────────────┐
| 4. TESTS verify the solution                            |
|    - Run test.sh (executes pytest)                      |
|    - Check: Does /app/output/summary.json exist?        |
|    - Check: Is JSON valid?                              |
|    - Check: Are all 4 regions present?                  |
|    - Check: Do totals match CSV data?                   |
|    - Write result to /logs/verifier/reward.txt          |
└─────────────────────────────────────────────────────────┘

                          ↓

┌─────────────────────────────────────────────────────────┐
| 5. PLATFORM calculates score                            |
|    - reward.txt = 1 → AI gets 1.0 score                 |
|    - reward.txt = 0 → AI gets 0.0 score                 |
|    - Run same task with 10 different AI models          |
|    - Calculate: Which model has highest success rate?   |
└─────────────────────────────────────────────────────────┘
```

What Happens Locally (For You)

```
# Build the Docker image
docker build -t terminus-csv-test -f environment/Dockerfile .
```

```
# Test the broken script (should crash)
docker run --rm terminus-csv-test python3 /app/aggregator.py
# Output: KeyError: 'Region'

# Test the oracle solution (should pass all tests)
docker run --rm \
  -v ./solution:/solution:ro \
  -v ./tests:/tests:ro \
  terminus-csv-test \
  bash -c "bash /solution/solve.sh && bash /tests/test.sh"
# Output: 10 passed, reward: 1.0

# Test a "do nothing" agent (should fail all tests)
docker run --rm -v ./tests:/tests:ro terminus-csv-test bash /tests/test.s
# Output: 10 failed, reward: 0.0
```

Expected Outcomes

Immediate Outcomes (Your Local Testing)

- ✅ Oracle solution passes 10/10 tests every time
- ✅ Do-nothing agent fails 10/10 tests every time
- ✅ Docker image builds in < 5 minutes
- ✅ All CI checks pass (Dockerfile format, file structure, etc.)

After Submission to Snorkel

1. **Fast Static Checks** (1-2 minutes)
    - File structure correct?
    - All required files present?
    - No syntax errors?
2. **CI + LLMaJ Checks** (10-15 minutes)
    - Dockerfile follows best practices?
    - Tests have good docstrings?
    - Instructions are clear?
    - No cheating possible?
3. **Agent Testing** (you run this locally first)

    - GPT-5.2 tries to solve it 10 times
    - Claude Opus tries to solve it 10 times
    - Success rate should be 30-70% (not too easy/hard)

4. **Peer Review** (human reviewer)
    - Is the task realistic?
    - Are instructions clear?
    - Is difficulty appropriate?

5. **Final Rating**
    - **Trivial** (>90% pass) → Rejected, must make harder
    - **Easy** (70-90% pass) → $250 payout
    - **Medium** (40-70% pass) → $250 payout
    - **Hard** (10-40% pass) → $250 payout
    - **Very Hard** (<10% pass) → May need to make easier

Long-Term Outcomes (Platform-Wide)

- Your task becomes part of the official Terminus benchmark dataset
- Researchers use it to compare AI models
- Future AI systems are trained to handle tasks like yours
- You contribute to advancing AI coding capabilities

Your Role as Task Creator

What You Do

1. **Design the challenge** — Pick a realistic debugging scenario
2. **Create broken code** — Write code with subtle, non-obvious bugs
3. **Write perfect solution** — Prove the task is solvable
4. **Write comprehensive tests** — Verify correctness without giving away answers
5. **Document clearly** — Help AI agents understand what to do
6. **Test locally** — Verify oracle passes, agents struggle appropriately
7. **Submit to platform** — Upload ZIP, pass CI/LLMaJ, get peer review
8. **Iterate if needed** — Fix issues reviewers find

What You Get Paid For

- ✅ Task accepted by peer reviewers → **$250**

- ✅ Each additional accepted task → **$250**
- ❌ Task rejected → $0 (but you can revise and resubmit)

Time Investment

- **First task:** 4-8 hours (learning curve)
- **Subsequent tasks:** 2-4 hours (process becomes routine)

Skills You Learn

- Docker container management
- pytest test framework
- bash scripting
- CI/CD best practices
- Technical writing
- Quality assurance

Common Questions

Q: Can the AI agent see my solution/solve.sh?
A: No. Only `instruction.md` and the broken code inside the container are visible.

Q: Why 4 bugs instead of 1?
A: Makes it harder for AI agents to solve quickly. Tests deeper reasoning ability.

Q: What if my task is too easy?
A: Platform rates it "Trivial" → rejected. You must add difficulty (more bugs, edge cases).

Q: What if my task is too hard?
A: Agents fail 100% → also rejected. You must make it solvable (reduce bugs, add hints).

Q: Can I use languages other than Python?
A: Yes! Terminus supports: Python, JavaScript, Go, Rust, Java, C++, Ruby, PHP, Shell scripts.

Q: How many tasks can I submit?
A: No limit, but Python tasks can't exceed 50% of the total dataset.

Q: What happens after I submit?
A: CI checks (auto) → LLMaJ checks (auto) → Peer review (human) → Acceptance + payment.

Next Steps

1. ✅ **Read this document thoroughly**
2. ✅ **Test your local setup** — Run oracle + tests, verify 10/10 pass
3. ⬜ **Package for submission** — Create ZIP file (see prompt.md Module 9)
4. ⬜ **Submit to platform** — Upload, check CI feedback, generate rubric
5. ⬜ **Wait for review** — Respond to feedback if needed
6. ⬜ **Get paid!** — Accepted tasks receive $250 payment

Resources

- **Terminus Platform:** https://snorkel-ai.github.io/Terminus-ECTraining-stateful/portal
- **Slack Support:** #terminus-2nd-edition-submission
- **Office Hours:** Monday, Wednesday, Friday (ask Snorkel staff)
- **Docker Docs:** https://docs.docker.com/get-started/
- **pytest Docs:** https://docs.pytest.org/en/7.4.x/

Congratulations! You now understand Project Terminus from end to end. 🎉